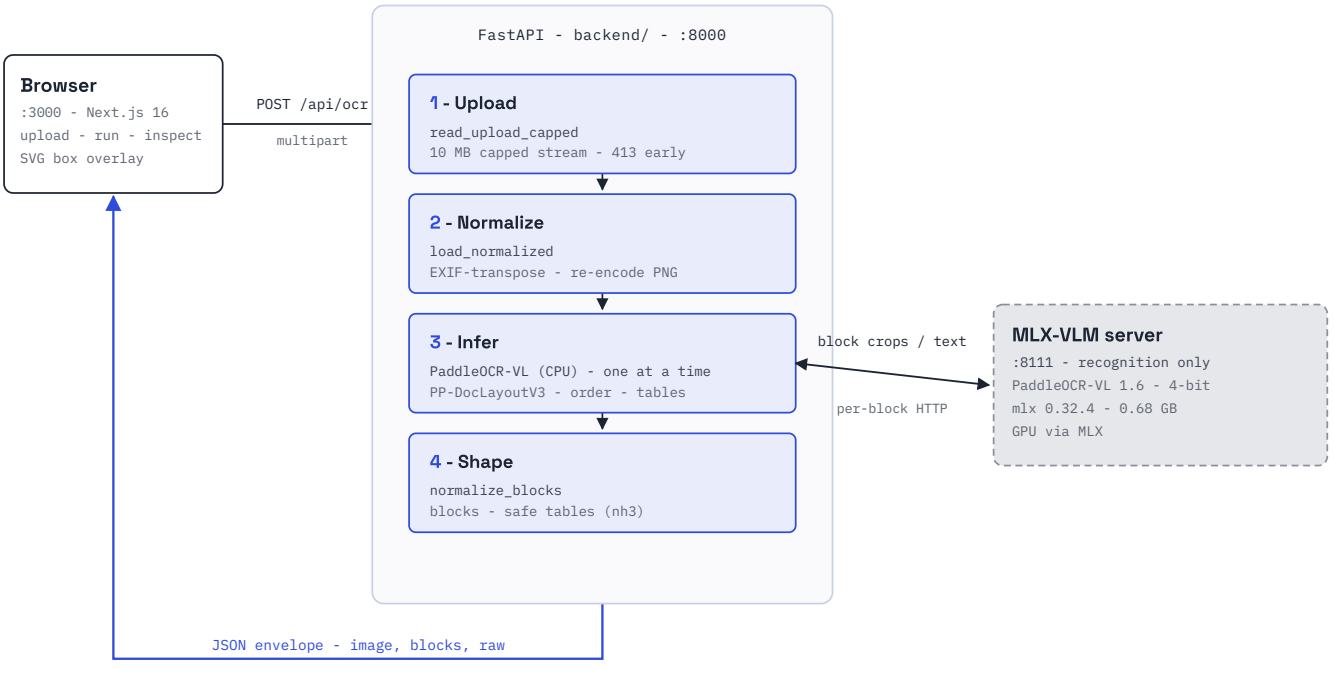
10 Invoice OCR Viewer

system architecture - PaddleOCR-VL layout (CPU) + MLX recognition - all local, Apple Silicon



One request, end to end. Layout and reading order run inside the backend on CPU; only recognition is delegated to the MLX server.

blue = stages inside the FastAPI backend - blue line = response path - dashed = separate process - white = browser

01 How a request flows

The backend owns everything between the upload and the JSON: validation, normalization, inference, and shaping. One `asyncio.Lock` serializes inference - one page at a time, no queue service.

1 - Upload

`read_upload_capped`

The multipart body is streamed in 1 MB chunks with a running 10 MB cap, so an oversize upload fails with 413 before it is ever buffered.

2 - Normalize

`load_normalized`

Decode by actual content (JPEG or PNG only, never by extension), min 50 px side, max 30 MP. EXIF-transpose, convert to RGB, re-encode as PNG. These exact bytes are what inference sees.

3 - Infer

`RealPipeline`

Probe the MLX server (`/v1/models`, 503 if down), then run the PaddleOCR-VL pipeline: PP-DocLayoutV3 layout on CPU, recognition delegated per block to the MLX server. Lazy init, reused across requests; temp files deleted in `finally`.

4 - Shape

`normalize_blocks`

Numpy arrays to plain lists, reading order falls back to `block_id` when null, blocks sorted. Table HTML sanitized server-side with an nh3 allowlist; the temp-file `input_path` is stripped.

02 Why the boxes always line up

The coordinate basis is closed by construction:

- The backend normalizes the image first and gives those exact PNG bytes to the pipeline.
- The same bytes come back to the browser as `image.data_url`.
- `block_bbox` values are therefore pixels of the displayed image - no conversion, no guessing.
- The overlay is one SVG with `viewBox="0 0 width height"` pinned over the `img`, so boxes stay aligned at any rendered size.
- Rotation and unwarping are disabled in the pipeline, so no hidden geometry sits between bbox and pixels.

03 Trust boundaries

OCR output and model HTML are treated as untrusted input:

- Uploads validated by decode, capped while streaming; CORS limited to localhost:3000.
- Block text renders as React text nodes (escaped), never as HTML.
- Table HTML is sanitized server-side (nh3, table-tag allowlist) before any `dangerouslySetInnerHTML`.
- Selection sync keys on stable `block_id`, never on text - documents repeat text.
- A failed run clears all previous results; no stale output, no fake success.

04 Stack and measured numbers

Everything below was measured on the demo machine (Apple M2, 16 GB). No accuracy or speed claims beyond these numbers.

LAYER	TECHNOLOGY	ROLE
Viewer	`Next.js 16.4.3 - React 19.3`	Upload, run, image + reading-order list, bidirectional selection, JSON download
API	`FastAPI 0.162 - Python 3.11 (uv)`	Validation, normalization, orchestration, one response envelope for every outcome
Layout	`paddleocr 3.7.0 - paddlepaddle 3.2.1 (CPU)`	PP-DocLayoutV3: regions, reading order, table structure
Recognition	`mlx-vlm 0.7.11 - mlx 0.32.4`	Serves `PaddleOCR-VL-1.6-4bit` (0.68 GB) on the GPU, recognition only
Latency	`12.6 - 21.4 s` per page	12 - 13 blocks on typical invoices and receipts; first request adds about 20 s one-time init
Tests	`24 + 6 + 4`	pytest (stub backend, no model), vitest, Playwright against a mocked API

Local only - no cloud APIs, no auth, no database. Generated from the invoice_ocr repository, 2026-11-01.